FlowDeck

Full Documentation

Version 1.0

Date May 2026

Status Production

Stack React · Express · PostgreSQL · Clerk

System architecture, tech stack, API reference, database schema, and feature documentation for the FlowDeck productivity platform.

FlowDeck — Full Documentation

A full-stack productivity web application for tracking goals, habits, tasks, focus sessions, and mood — all in one unified daily workspace.

Table of Contents

1. Project Overview

FlowDeck is a personal productivity cockpit. Users can set and track quantitative/milestone/habit goals, manage prioritized to-do lists, log daily habits with streak tracking, run Pomodoro-style focus sessions, record mood entries, and review the week through charts and a reflection journal — all behind a single sign-in.

Core modules:

Module	Description
|D|a|s|h|b|o|a|r|d|	|D|a|i|l|y| |s|u|m|m|a|r|y| |a|c|r|o|s|s| |a|l|l| |m|o|d|u|l|e|s|
|G|o|a|l|s|	|Q|u|a|n|t|i|t|a|t|i|v|e| |/| |m|i|l|e|s|t|o|n|e| |/| |h|a|b|i|t| |g|o|a|l|s| |w|i|t|h| |p|r|o|g|r|e|s|s| |b|a|r|s| |a|n|d| |m|i|l|e|s|t|o|n|e| |t|o|a|s|t|s|
|T|a|s|k|s|	|P|r|i|o|r|i|t|i|z|e|d| |t|o|-|d|o| |l|i|s|t|s| |w|i|t|h| |s|u|b|t|a|s|k|s|,| |r|e|c|u|r|r|e|n|c|e| |t|a|g||f|i|l|t|e|r|i|n|g|
|H|a|b|i|t|s|	|D|a|i|l|y| |h|a|b|i|t| |t|r|a|c|k|e|r| |w|i|t|h| |s|t|r|e|a|k| |c|a|l|c|u|l|a|t|i|o|n| |g|r|a|c|e| |d|a|y|s| |h|e|a|t|m|a|p| |c|a|l|l|e|n|d|a|r|
|F|o|c|u|s|	|P|o|m|o|d|o|r|o|-|s|t|y|l|e| |t|i|m|e|r| |(|2|5|/|5| |+| |l|o|n|g| |b|r|e|a|k|)| |s|e|s|s|i|o|n|s| |s|a|v|e|d| |t|o| |D|B|
|M|o|o|d|	|D|a|i|l|y| |m|o|o|d| |l|o|g| |(|1|–|5| |s|c|a|l|e|)| |w|i|t|h| |o|p|t|i|o|n|a|l| |n|o|t|e|s|
|C|a|t|e|g|o|r|i|e|s|	|C|o|l|o|r|-|c|o|d|e|d| |b|u|c|k|e|t|s| |s|h|a|r|e|d| |a|c|r|o|s|s| |g|o|a|l|s| |a|n|d| |t|a|s|k|s|
|T|a|g|s|	|F|r|e|e|-|f|o|r|m| |l|a|b|e|l|s| |f|o|r| |c|r|o|s|s|-|c|u|t|t|i|n|g| |t|a|s|k| |o|r|g|a|n|i|z|a|t|i|o|n|
|W|e|e|k|l|y| |R|e|v|i|e|w|	|W|e|e|k|l|y| |s|t|a|t|s| |d|a|s|h|b|o|a|r|d| |w|i|t|h| |c|h|a|r|t|s| |a|n|d| |a| |r|e|f|l|e|c|t|i|o|n| |j|o|u|r|n|a|l|

2. Tech Stack

Runtime & Build

Concern	Tool
|P|a|c|k|a|g|e| |m|a|n|a|g|e|r|	|p|n|p|m| |9| |(|w|o|r|k|s|p|a|c|e|s|)|
|N|o|d|e|.|j|s|	|v|2|4|
|T|y|p|e|S|c|r|i|p|t|	|5|.|9| |(|s|t|r|i|c|t| |m|o|d|e| |t|h|r|o|u|g|h|o|u|t|)|
|B|u|i|l|d| |(|s|e|r|v|e|r|)|	|e|s|b|u|i|l|d| |—| |C|J|S| |b|u|n|d|l|e|
|B|u|i|l|d| |(|c|l|i|e|n|t|)|	|V|i|t|e| |6|

Frontend

Concern	Library
|U|I| |f|r|a|m|e|w|o|r|k|	|R|e|a|c|t| |1|9|
|R|o|u|t|i|n|g|	|W|o|u|t|e|r|
|S|e|r|v|e|r| |s|t|a|t|e|	|T|a|n|S|t|a|c|k| |Q|u|e|r|y| |v|5|
|S|t|y|l|i|n|g|	|T|a|i|l|w|i|n|d| |C|S|S| |v|4|
|C|o|m|p|o|n|e|n|t| |l|i|b|r|a|r|y|	|s|h|a|d|c|n|/|u|i| |(|R|a|d|i|x| |U|I| |p|r|i|m|i|t|i|v|e|s|)|
|C|h|a|r|t|s|	|R|e|c|h|a|r|t|s|
|N|o|t|i|f|i|c|a|t|i|o|n|s|	|S|o|n|n|e|r| |(|t|o|a|s|t|)|
|I|c|o|n|s|	|L|u|c|i|d|e| |R|e|a|c|t|
|A|u|t|h| |(|c|l|i|e|n|t|)|	|@|c|l|e|r|k|/|r|e|a|c|t|

Backend

Concern	Library
|H|T|T|P| |f|r|a|m|e|w|o|r|k|	|E|x|p|r|e|s|s| |5|
|A|u|t|h| |(|s|e|r|v|e|r|)|	|@|c|l|e|r|k|/|e|x|p|r|e|s|s|
|O|R|M|	|D|r|i|z|z|l|e| |O|R|M|
|D|a|t|a|b|a|s|e|	|P|o|s|t|g|r|e|S|Q|L| |(|R|e|p|l|i|t|-|m|a|n|a|g|e|d|)|
|V|a|l|i|d|a|t|i|o|n|	|Z|o|d| |v|4| |+| |d|r|i|z|z|l|e|-|z|o|d|
|L|o|g|g|i|n|g|	|P|i|n|o| |(|v|i|a| |r|e|q|.|l|o|g| |i|n| |h|a|n|d|l|e|r|s|)|

API Contract

Concern	Tool
|S|p|e|c| |f|o|r|m|a|t|	|O|p|e|n|A|P|I| |3|.|0| |Y|A|M|L|
|C|o|d|e| |g|e|n|e|r|a|t|i|o|n|	|O|r|v|a|l|
|G|e|n|e|r|a|t|e|d| |o|u|t|p|u|t|s|	|R|e|a|c|t| |Q|u|e|r|y| |h|o|o|k|s| |(|l|i|b|/|a|p|i|-|c|l|i|e|n|t|-|r|e|a|c|t|)| |+| |Z|o|d| |v|a|l|i|d|a|t|o|r|s| |(|l|i|b|/|a|p|i|-|z|o|d|)|

3. System Architecture

```
Browser
  %
  %   HTTPS (Replit proxy — path-based routing)
  %
  % % %  /          !’  Vite dev server  (artifacts/flowdeck)   port $PORT
  % % %  /api       !’  Express server   (artifacts/api-server)  port $PORT
       % % %  /api/__clerk  !’  Clerk proxy (avoids 3rd-party cookie issues)
```

Request lifecycle

1. User loads the Rea

ct
app

2. Clerk SDK initialises and authenticates the user via the proxied /api/__clerk path.
3. ClerkQueryClientCacheInvalidator in App.tsx registers getToken() so every TanStack Query fetch automatically injects a Bearer <JWT> header.
4. React Query hoo

ks (

5. Express routes validate the JWT with @clerk/express, extract userId via requireAuth middleware, and return JSON.
6. Drizzle ORM queries PostgreSQL — all queries are scoped to userId so users

nev
er

Code generation pipeline

```
lib/api-spec/openapi.yaml
        %
        %¼  pnpm --filter @workspace/api-spec run codegen
        %
        % % %  lib/api-client-react/src/generated/api.ts   (React Query hooks)
        % % %  lib/api-zod/src/generated/                  (Zod validators for server)
```

Any change to the OpenAPI spec must be followed by codegen before typechecking.

4. Repository Structure

```
/
% % %  artifacts/
%    % % %  api-server/               Express API (Node.js, port $PORT, path /api)
%    %    % % %  src/
%    %        % % %  app.ts            Express app, Clerk middleware, CORS
%    %        % % %  routes/           One file per resource
%    %        % % %  middlewares/
%    %            % % %  requireAuth.ts        Extracts Clerk userId
%    %            % % %  clerkProxyMiddleware.ts
%    %
%    % % %  flowdeck/                 React + Vite frontend (port $PORT, path /)
%        % % %  src/
%            % % %  App.tsx           ClerkProvider, Wouter router, QueryClientProvider
%            % % %  pages/           One file per page/route
%            % % %  components/      Layout, sidebar nav, shared UI
%            % % %  lib/
%                % % %  queryClient.ts
%                % % %  goalMilestones.ts     Milestone toast logic (localStorage)
%
% % %  lib/
%    % % %  db/src/schema/           Drizzle ORM table definitions (source of truth)
%    % % %  api-spec/openapi.yaml    OpenAPI spec (source of truth for API contract)
%    % % %  api-client-react/        Generated React Query hooks + custom-fetch.ts
%    % % %  api-zod/                 Generated Zod validators (used by API server)
%
% % %  scripts/                     Shared utility scripts
% % %  pnpm-workspace.yaml          Workspace catalog + overrides
% % %  tsconfig.json                Root solution file (composite libs only)
% % %  tsconfig.base.json           Shared strict TS defaults
```

TypeScript model:

- li
b/
*
p
ac
ka
g
es
a
re
c
o
m
p

os
ite

- artifacts/* are leaf packages — checked with tsc --noEmit, never emit.
- Root tsconfig.json

5. Database Schema

All tables include a userId (Clerk user ID string) that scopes every query. No row is ever visible across users.

categories

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|I|d|	|t|e|x|t|	|C|l|e|r|k| |u|s|e|r| |I|D|
|n|a|m|e|	|t|e|x|t|	
|i|c|o|n|	|t|e|x|t|	|d|e|f|a|u|l|t| |"|f|o|l|d|e|r|"|
|c|o|l|o|r|	|t|e|x|t|	|h|e|x|,| |d|e|f|a|u|l|t| |"|#|6|3|6|6|f|1|"|
|d|e|s|c|r|i|p|t|i|o|n|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

goals

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|l|d|	|t|e|x|t|	
|t|i|t|l|e|	|t|e|x|t|	
|d|e|s|c|r|i|p|t|i|o|n|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|c|a|t|e|g|o|r|y|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |c|a|t|e|g|o|r|i|e|s|
|g|o|a|l|T|y|p|e|	|t|e|x|t|	|q|u|a|n|t|i|t|a|t|i|v|e| ||
|s|t|a|t|u|s|	|t|e|x|t|	|a|c|t|i|v|e| ||
|p|r|i|o|r|i|t|y|	|t|e|x|t|	|h|i|g|h| ||
|t|a|r|g|e|t|V|a|l|u|e|	|r|e|a|l|	|n|u|l|l|a|b|l|e| |—| |u|s|e|d| |f|o|r| |q|u|a|n|t|i|t|a|t|i|v|e| |g|o|a|l|s|
|c|u|r|r|e|n|t|V|a|l|u|e|	|r|e|a|l|	|d|e|f|a|u|l|t| |0|
|s|t|a|r|t|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|,| |n|u|l|l|a|b|l|e|
|t|a|r|g|e|t|E|n|d|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|,| |n|u|l|l|a|b|l|e|
|n|o|t|e|s|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|c|r|e|a|t|e|d|A|t| |/| |u|p|d|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

tasks

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|l|d|	|t|e|x|t|	
|t|i|t|l|e|	|t|e|x|t|	
|n|o|t|e|s|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|g|o|a|l|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |g|o|a|l|s|,| |n|u|l|l|a|b|l|e|
|c|a|t|e|g|o|r|y|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |c|a|t|e|g|o|r|i|e|s|,| |n|u|l|l|a|b|l|e|
|p|r|i|o|r|i|t|y|	|t|e|x|t|	|h|i|g|h| ||
|s|t|a|t|u|s|	|t|e|x|t|	|p|e|n|d|i|n|g| ||
|d|u|e|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|,| |n|u|l|l|a|b|l|e|
|r|e|c|u|r|r|e|n|c|e|	|t|e|x|t|	|d|a|i|l|y| ||
|e|s|t|i|m|a|t|e|d|M|i|n|u|t|e|s|	|i|n|t|e|g|e|r|	|n|u|l|l|a|b|l|e|
|c|o|m|p|l|e|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	|s|e|t| |w|h|e|n| |s|t|a|t|u|s| |!'| |c|o|m|p|l|e|t|e|d|
|s|o|r|t|O|r|d|e|r|	|i|n|t|e|g|e|r|	|d|e|f|a|u|l|t| |0|
|c|r|e|a|t|e|d|A|t| |/| |u|p|d|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

subtasks

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|t|a|s|k|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |t|a|s|k|s|
|u|s|e|r|l|d|	|t|e|x|t|	
|t|i|t|l|e|	|t|e|x|t|	
|c|o|m|p|l|e|t|e|d|	|b|o|o|l|e|a|n|	|d|e|f|a|u|l|t| |f|a|l|s|e|
|s|o|r|t|O|r|d|e|r|	|i|n|t|e|g|e|r|	|d|e|f|a|u|l|t| |0|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

task_tags (junction)

Column	Type	Notes
|t|a|s|k|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |t|a|s|k|s|,| |c|o|m|p|o|s|i|t|e| |P|K|
|t|a|g|l|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |t|a|g|s|,| |c|o|m|p|o|s|i|t|e| |P|K|

tags

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|l|d|	|t|e|x|t|	
|n|a|m|e|	|t|e|x|t|	
|c|o|l|o|r|	|t|e|x|t|	|h|e|x|,| |d|e|f|a|u|l|t| |"|#|6|3|6|6|f|1|"|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

habits

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|I|d|	|t|e|x|t|	
|n|a|m|e|	|t|e|x|t|	
|c|a|t|e|g|o|r|y|I|d|	|i|n|t|e|g|e|r|	|n|u|l|l|a|b|l|e|
|g|o|a|l|I|d|	|i|n|t|e|g|e|r|	|n|u|l|l|a|b|l|e|
|f|r|e|q|u|e|n|c|y|	|t|e|x|t|	|d|a|i|l|y| ||
|t|a|r|g|e|t|V|a|l|u|e|	|r|e|a|l|	|n|u|l|l|a|b|l|e|
|i|c|o|n|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|c|o|l|o|r|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|m|o|t|i|v|a|t|i|o|n|N|o|t|e|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|g|r|a|c|e|D|a|y|s|P|e|r|W|e|e|k|	|i|n|t|e|g|e|r|	|d|e|f|a|u|l|t| |0| |—| |s|t|r|e|a|k| |f|r|e|e|z|e|a|l|l|o|w|a|n|c|e|
|i|s|A|r|c|h|i|v|e|d|	|b|o|o|l|e|a|n|	|d|e|f|a|u|l|t| |f|a|l|s|e|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

habit_logs

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|I|d|	|t|e|x|t|	
|h|a|b|i|t|I|d|	|i|n|t|e|g|e|r|	|F|K| |!'| |h|a|b|i|t|s|
|l|o|g|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|
|v|a|l|u|e|	|r|e|a|l|	|n|u|l|l|a|b|l|e| |—| |f|o|r| |q|u|a|n|t|i|t|a|t|i|v|e| |h|a|b|i|t|s|
|n|o|t|e|s|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

mood_logs

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|I|d|	|t|e|x|t|	
|m|o|o|d|	|i|n|t|e|g|e|r|	|1|–|5|
|n|o|t|e|s|	|t|e|x|t|	|n|u|l|l|a|b|l|e|
|l|o|g|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

focus_sessions

Column	Type	Notes
|i|d|	|s|e|r|i|a|l| |P|K|	
|u|s|e|r|I|d|	|t|e|x|t|	
|t|a|s|k|I|d|	|i|n|t|e|g|e|r|	|n|u|l|l|a|b|l|e| |—| |l|i|n|k|e|d| |t|a|s|k|
|d|u|r|a|t|i|o|n|M|i|n|u|t|e|s|	|i|n|t|e|g|e|r|	
|s|e|s|s|i|o|n|T|y|p|e|	|t|e|x|t|	|p|o|m|o|d|o|r|o| || |s|h|o|r|t|_|b|r|e|
|s|e|s|s|i|o|n|D|a|t|e|	|t|e|x|t|	|I|S|O| |d|a|t|e| |s|t|r|i|n|g|
|c|r|e|a|t|e|d|A|t|	|t|i|m|e|s|t|a|m|p|t|z|	

6. API Reference

All endpoints are prefixed /api and require a valid Clerk Bearer token. The userId is never passed by the client — the requireAuth middleware reads it from the Clerk session and injects it server-side.

Goals

Method	Path	Description
|G|E|T|	|/|a|p|i|/|g|o|a|l|s|	|L|i|s|t| |a|l|l| |g|o|a|l|s| |f|o|r| |t|h|e| |u|s|e|r|
|P|O|S|T|	|/|a|p|i|/|g|o|a|l|s|	|C|r|e|a|t|e| |a| |g|o|a|l|
|G|E|T|	|/|a|p|i|/|g|o|a|l|s|/|:|i|d|	|G|e|t| |a| |s|i|n|g|l|e| |g|o|a|l|
|P|A|T|C|H|	|/|a|p|i|/|g|o|a|l|s|/|:|i|d|	|U|p|d|a|t|e| |a| |g|o|a|l|
|D|E|L|E|T|E|	|/|a|p|i|/|g|o|a|l|s|/|:|i|d|	|D|e|l|e|t|e| |a| |g|o|a|l|
|G|E|T|	|/|a|p|i|/|g|o|a|l|s|/|:|i|d|/|p|r|o|g|r|e|s|s|	|C|o|m|p|u|t|e|d| |p|r|o|g|r|e|s|s| |d|e|t|a|i|l| |(|p|e|r|c|e|n|t| | |m|i|l|e|s|t|o|n|e| |c|o|u|n|t|
|P|O|S|T|	|/|a|p|i|/|g|o|a|l|s|/|:|i|d|/|c|o|m|p|l|e|t|e|	|M|a|r|k| |a| |g|o|a|l| |a|s| |c|o|m|p|l|e|t|e|d|

Tasks

Method	Path	Description
|G|E|T|	|/|a|p|i|/|t|a|s|k|s|/|t|o|d|a|y|	|T|a|s|k|s| |d|u|e| |t|o|d|a|y|
|G|E|T|	|/|a|p|i|/|t|a|s|k|s|	|L|i|s|t| |a|l|l| |t|a|s|k|s| |(|s|u|p|p|o|r|t|s| |?| g|o|a|l|l|d|=| | |?|s|t|a|t|u|s|=|)|
|P|O|S|T|	|/|a|p|i|/|t|a|s|k|s|	|C|r|e|a|t|e| |a| |t|a|s|k|
|G|E|T|	|/|a|p|i|/|t|a|s|k|s|/|:|i|d|	|G|e|t| |a| |s|i|n|g|l|e| |t|a|s|k|
|P|A|T|C|H|	|/|a|p|i|/|t|a|s|k|s|/|:|i|d|	|U|p|d|a|t|e| |a| |t|a|s|k|
|D|E|L|E|T|E|	|/|a|p|i|/|t|a|s|k|s|/|:|i|d|	|D|e|l|e|t|e| |a| |t|a|s|k|
|P|O|S|T|	|/|a|p|i|/|t|a|s|k|s|/|:|i|d|/|c|o|m|p|l|e|t|e|	|C|o|m|p|l|e|t|e| |a| |t|a|s|k| |—| |a|u|t|o|-|s|p|a|w|n|s| |n|e|x|t| |o|c|c|u|r|r|e|n|c|e| |i|f| |r|e|c|u|r|r|e|n|c|e| |i|s| |s|e|t|

Subtasks

Method	Path	Description
|G|E|T|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|s|u|b|t|a|s|k|s|	|L|i|s|t| |s|u|b|t|a|s|k|s| |f|o|r| |a| |t|a|s|k|
|P|O|S|T|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|s|u|b|t|a|s|k|s|	|C|r|e|a|t|e| |a| |s|u|b|t|a|s|k|
|P|A|T|C|H|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|s|u|b|t|a|s|k|s|/|:|i|d|	|U|p|d|a|t|e| |a| |s|u|b|t|a|s|k|
|D|E|L|E|T|E|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|s|u|b|t|a|s|k|s|/|:|i|d|	|D|e|l|e|t|e| |a| |s|u|b|t|a|s|k|
|P|O|S|T|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|s|u|b|t|a|s|k|s|/|:|i|d|/|t|o|g|g|l|e|	|T|o|g|g|l|e| |a| |s|u|b|t|a|s|k|'|s| |c|o|m|p|l|e|t|e|d| |s|t|a|t|e|

Task Tags

Method	Path	Description
|G|E|T|	|/|a|p|i|/|t|a|s|k|-|t|a|g|s|	|A|l|l| |t|a|s|k|-|t|a|g| |a|s|s|o|c|i|a|t|i|o|n|s| |f|o|r| |t|h|e| |u|s|e|r| |(|s|i|n|g|l|e| |a|g|r|e|
|G|E|T|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|t|a|g|s|	|T|a|g|s| |f|o|r| |a| |s|p|e|c|i|f|i|c| |t|a|s|k|
|P|U|T|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|t|a|g|s|/|:|t|a|g|l|d|	|A|s|s|i|g|n| |a| |t|a|g| |t|o| |a| |t|a|s|k|
|D|E|L|E|T|E|	|/|a|p|i|/|t|a|s|k|s|/|:|t|a|s|k|l|d|/|t|a|g|s|/|:|t|a|g|l|d|	|R|e|m|o|v|e| |a| |t|a|g| |f|r|o|m| |a| |t|a|s|k|

Tags

Method	Path	Description
|G|E|T|	|/|a|p|i|/|t|a|g|s|	|L|i|s|t| |a|l|l| |t|a|g|s|
|P|O|S|T|	|/|a|p|i|/|t|a|g|s|	|C|r|e|a|t|e| |a| |t|a|g|
|D|E|L|E|T|E|	|/|a|p|i|/|t|a|g|s|/|:|i|d|	|D|e|l|e|t|e| |a| |t|a|g|

Habits

Method	Path	Description
|G|E|T|	|/|a|p|i|/|h|a|b|i|t|s|	|L|i|s|t| |a|l|l| |a|c|t|i|v|e| |h|a|b|i|t|s|
|P|O|S|T|	|/|a|p|i|/|h|a|b|i|t|s|	|C|r|e|a|t|e| |a| |h|a|b|i|t|
|G|E|T|	|/|a|p|i|/|h|a|b|i|t|s|/|:|i|d|	|G|e|t| |a| |s|i|n|g|l|e| |h|a|b|i|t|
|P|A|T|C|H|	|/|a|p|i|/|h|a|b|i|t|s|/|:|i|d|	|U|p|d|a|t|e| |a| |h|a|b|i|t|
|D|E|L|E|T|E|	|/|a|p|i|/|h|a|b|i|t|s|/|:|i|d|	|D|e|l|e|t|e| |(|a|r|c|h|i|v|e|)| |a| |h|a|b|i|t|
|G|E|T|	|/|a|p|i|/|h|a|b|i|t|s|/|:|i|d|/|s|t|r|e|a|k|s|	|C|u|r|r|e|n|t| |s|t|r|e|a|k|,| |l|o|n|g|e|s|t| |s|t|r|e|a|k| | |g|r|a|c|e|-|d|a|y| |l|o|g|i|c|
|G|E|T|	|/|a|p|i|/|h|a|b|i|t|s|/|:|i|d|/|h|e|a|t|m|a|p|	|1|2|-|m|o|n|t|h| |l|o|g| |h|e|a|t|m|a|p| |d|a|t|a|
|G|E|T|	|/|a|p|i|/|h|a|b|i|t|-|l|o|g|s|	|L|i|s|t| |h|a|b|i|t| |l|o|g|s|
|P|O|S|T|	|/|a|p|i|/|h|a|b|i|t|-|l|o|g|s|	|L|o|g| |a| |h|a|b|i|t| |f|o|r| |a| |d|a|t|e|
|D|E|L|E|T|E|	|/|a|p|i|/|h|a|b|i|t|-|l|o|g|s|/|:|i|d|	|D|e|l|e|t|e| |a| |h|a|b|i|t| |l|o|g| |(|u|n|-|l|o|g|)|

Mood Logs

Method	Path	Description
|G|E|T|	|/|a|p|i|/|m|o|o|d|-|l|o|g|s|/|t|o|d|a|y|	|T|o|d|a|y|'|s| |m|o|o|d| |e|n|t|r|y|
|G|E|T|	|/|a|p|i|/|m|o|o|d|-|l|o|g|s|	|L|i|s|t| |a|l|l| |m|o|o|d| |l|o|g|s|
|P|O|S|T|	|/|a|p|i|/|m|o|o|d|-|l|o|g|s|	|C|r|e|a|t|e| |a| |m|o|o|d| |l|o|g|

Focus Sessions

Method	Path	Description
|G|E|T|	|/|a|p|i|/|f|o|c|u|s|-|s|e|s|s|i|o|n|s|	|L|i|s|t| |a|l|l| |f|o|c|u|s| |s|e|s|s|i|o|n|s|
|P|O|S|T|	|/|a|p|i|/|f|o|c|u|s|-|s|e|s|s|i|o|n|s|	|S|a|v|e| |a| |c|o|m|p|l|e|t|e|d| |s|e|s|s|i|o|n|

Categories

Method	Path	Description
|G|E|T|	|/|a|p|i|/|c|a|t|e|g|o|r|i|e|s|	|L|i|s|t| |a|l|l| |c|a|t|e|g|o|r|i|e|s|
|P|O|S|T|	|/|a|p|i|/|c|a|t|e|g|o|r|i|e|s|	|C|r|e|a|t|e| |a| |c|a|t|e|g|o|r|y|
|G|E|T|	|/|a|p|i|/|c|a|t|e|g|o|r|i|e|s|/|:|i|d|	|G|e|t| |a| |c|a|t|e|g|o|r|y|
|P|A|T|C|H|	|/|a|p|i|/|c|a|t|e|g|o|r|i|e|s|/|:|i|d|	|U|p|d|a|t|e| |a| |c|a|t|e|g|o|r|y|
|D|E|L|E|T|E|	|/|a|p|i|/|c|a|t|e|g|o|r|i|e|s|/|:|i|d|	|D|e|l|e|t|e| |a| |c|a|t|e|g|o|r|y|

Dashboard

Method	Path	Description
|G|E|T|	|/|a|p|i|/|d|a|s|h|b|o|a|r|d|/|s|u|m|m|a|r|y|	|D|a|i|l|y| |s|u|m|m|a|r|y| |(|h|a|b|i|t|s| |t|o| d|a|y| |-|t|a|s|k|s| |d|u|e| |a|c|t|i|v|e| |g|o|
|G|E|T|	|/|a|p|i|/|d|a|s|h|b|o|a|r|d|/|w|e|e|k|l|y|-|o|v| e|r|v|i|e|w|	|7|-|d|a|y| |o|v|e|r|v|i|e|w| |a|g|g|r|e|g|a|t|i| o|n|
|G|E|T|	|/|a|p|i|/|d|a|s|h|b|o|a|r|d|/|p|r|o|d|u|c|t|i|v|i| t|y|-|s|c|o|r|e|	|W|e|i|g|h|t|e|d| |d|a|i|l|y| |p|r|o|d|u|c|t|i|v|i| t|y| |s|c|o|r|e|
|G|E|T|	|/|a|p|i|/|d|a|s|h|b|o|a|r|d|/|u|p|c|o|m|i|n|g|	|U|p|c|o|m|i|n|g| |t|a|s|k|s| |b|y| |d|u|e| |d|a| t|e|
|G|E|T|	|/|a|p|i|/|d|a|s|h|b|o|a|r|d|/|w|e|e|k|l|y|-|r|e| v|i|e|w|	|F|u|l|l| |7|-|d|a|y| |r|e|v|i|e|w| |p|a|y|l|o|a| d| |(|p|e|r|-|d|a|y| |b|r|e|a|k|d|o|w|n| |+| |t|o| t|a|l|s| |+| |s|c|o|r|e|)|

7. Frontend Pages & Features

/ — Landing Page

Public marketing page. Redirects authenticated users to /dashboard.

/dashboard — Dashboard

- Daily habit compl

eti

- Today's tasks (due today)
- Active goals with progress
- Today's mood entry/prompt
- Focus time logged today

- Quick access links to all modules

/goals — Goals

- Create, edit, delete goals (quantitative/milestone/habit types)
- Status filter: all / active / completed / paused
- Progress

b
er

- Mi
le
st
o
n
e
b
a
d
g
e
o
n
ca
rd
(Ø<ß¯
2
5
%
/
&¡ 5
0
%
/
Ø=Ý% 7
5
%
/
Ø<ßÆ C
o
m
pl
et
e)
-

Q
ui

- Mi
le
st

p
or

- Dr ill- in to G o al D et ail vi a th e !’ b ut to n

/goals/:id — Goal Detail

- F ull g o al m et a d at a
- Li nk e d

task

- Linked habit logs for habit-type goals

/tasks — Tasks

- Full task list with expand/collapse panels
- Filter by status

(
p

- Tag filter bar — colored chips for each tag; shows count of tagged task

s;
filt

- Tag pills on task cards showing assigned tags
- Inline tag manager in expanded panel — toggle

ta
ge
•
S
u
bt
as
ks
—
in
lin
e
ch
ec
kli
st
in
ex
p
a
n
d
e
d
p
a
n
el
wi
th
a
pr
o
gr
es
s
b
ar
; a
d
d/
ch
ec
k/
d
el
et
e
su
bt
as
ks
w
ith
o
ut
le
av
in
g
th
e
p
a
g
e

- Task recurrence — daily / weekly / monthly; completing a recurring task auto-spawns the next occurrence with the correct due date
- Recurrence badge on task cards
- Due date, priority badge, estimated time display

/habits — Habits

- Habit cards with today's log toggle (one

cli
ck

- C
ur
re
nt
 st
re
ak
 w
ith
 Ø=Ý% i
n
di
ca
to
r
- St
re
ak
 fr
e
ez
e
(g
ra
ce
 d
ay
s)
—
 h
a
bit
s
wi
th
 g
ra
ce
D
ay
s
P
er
W
e
ek

>
0
sh
o
w
a
Ø=Þá s
hi
el
d;
mi
ss
e

d
d

- C ol or a n d i co n p er h a bit
- Ar ch iv e h a bit s

/habits/:id — Habit Detail

- F ull st re ak st at s: cu rr e nt st re

akl

- Grace-day allowance banner (amber if shield is active, teal if configured)
- 12-month heatmap calendar

(d

- Log/un-log for any past date

/focus — Focus Timer

- Pomodoro-style countdown: 25-minute work / 5-minute short

br
o

- Visualring countdown
- Links session to a specific task
- On completion, POSTs a focus_sessions record

to
t

/mood (embedded in Dashboard)

- 1 – 5 mood scale with emoji labels (Ø=Þ Rough! Ø=Þ Great)
- Optional notes
- One entry per day; subsequent entries u

ps
er

/categories — Categories

- C
ol
or

+ i
co
n
p
er
c
at
e
g
or
y
- U
se
d
as

a
filt
er
/b
uc
ke
t
o
n
b
ot
h
g
o
al
s
a
n
d
ta
sk
s

/settings — Settings

- U
se
r
pr
ef
er
e
nc
es
p
a
n
el

/weekly-review — Weekly Review

A complete end-of-week analysis page covering the rolling 7-day window ending today.

Stats row:

- Tasks completed this week
- Total focus time (formatted `Xh Ym`)
- Habit completion rate (%) — color-coded green "e 70%, yellow "e 40%, red below
- Average

m

Weekly score — weighted composite: 40% habit rate + 30% focus presence (1 pomodoro/day = 100%) + 30% mood. Rated Excellent / Strong / Decent / Needs work.

Top habit highlight — habit with the longest current streak this week.

Goals overview — total / active / completed counts with a completion progress bar.

Four charts:

- Daily habit completion % (bar, color-coded by intensity)
- Focus minutes per day (bar with a 25-mi

nu
te

- Mood trend (line chart with colored dots + emoji legend)
- Tasks completed per day (bar)

Day-by-day breakdown table — all 7 days with today highlighted, showing mood, focus, habits, and tasks for each day.

Open tasks callout — amber-bordered panel listing overdue tasks (red) and tasks due this week (amber), each linking to the Tasks page. Hidden when nothing is pending.

Weekly reflection — free-form textarea at the bottom. Auto-saves to localStorage 800ms after typing stops, shows a "Saved ' " indicator, and persists the last 12 weeks of notes keyed by weekStart date.

8. Authentication

FlowDeck uses Clerk for user authentication.

How it works

1. ClerkProvider wraps the entire React app in App.tsx.
2. Clerk JS calls are proxied through /api/__clerk (the clerkProxyMiddleware in Express) so auth and API requests share the same origin — this avoids third-party cookie issues in modern brow

sers

3. C lerk Que ryCl ient Cac heIn vali dato r re gist ers use Aut h(). get Tok en via set Aut hTo ken Gett er. Eve ry API req uest ma de t hro ugh the gen erat ed h ook s au tom atic ally i nclu des Aut hori zati on: Bea rer <tok en>.
4. On the serv er, r equi reA uth mid dle war e (@cl erk/ expr ess) vali

date

Route protection

- Public routes: /, /sign-in, /sign-up
- All other routes: wrap

p
e

9. Key Architectural Decisions

Contract-first API

The OpenAPI spec (lib/api-spec/openapi.yaml) is the single source of truth for the API contract. It drives:

- R
e
ac
t
Q
u
er
y
h
o
ok
s (
vi
a
Or
va
l c
o
d
e
g
e

n)

- Zod validators (via Orval codegen) used by the server to validate request/response shapes

This means any API change starts with editing the spec and re-running codegen — never the other way around.

Composite libs, leaf artifacts

lib/ packages emit TypeScript declarations (tsc --build) so they can be imported by both the frontend and backend. artifacts/ packages never emit — they are checked only with --noEmit. This prevents type portability issues (TS2742) in a monorepo.

Goal progress milestones (25/50/75/100%) are detected and persisted entirely client-side using localStorage. This avoids a DB schema change and keeps the feature stateless on the server. The hook skips toasts on the initial page load (to avoid spamming stale data on sign-in) and resets seen milestones if progress drops below a threshold (e.g. user lowers currentValue).

Weekly reflection in localStorage

The weekly reflection note is stored in localStorage keyed by weekStart date. Up to 12 weeks of notes are retained. This is intentional — reflection notes are personal, client-local, and don't need to sync across devices for a single-user productivity tool.

Task recurrence on the server

When POST /tasks/:id/complete is called on a recurring task, the server immediately creates the next task occurrence with the correct due date (+1 day / +7 days / +1 month), copying all fields except completedAt and status. This keeps recurrence logic server-side and prevents duplicates.

Streak freeze (grace days)

Habits support a graceDaysPerWeek field. The streak calculation algorithm counts how many days in the trailing 7 days are missed, and if the count is within the allowance it does not break the streak. This is computed on the fly in the GET /habits/:id/streaks route — no extra DB table needed.

Single tag aggregation endpoint

GET /api/task-tags returns all task-tag associations for a user in a single call. This is used by the Tasks page to display tag pills on every card and populate the tag filter bar without making N per-task requests.

10. Data Flow

Completing a recurring task

```
User clicks "Complete" on task card
  !’ POST /api/tasks/:id/complete
  !’ Server marks task completed (completedAt = now)
  !’ If recurrence is set, server creates next task with new dueDate
  !’ Response returns { completed: Task, next: Task | null }
  !’ React Query invalidates tasks query
  !’ UI refreshes with new pending task visible
```

Logging a habit

```
User toggles habit on Habits page
  !’ If not logged: POST /api/habit-logs { habitId, logDate: today }
  !’ If already logged: DELETE /api/habit-logs/:id
  !’ React Query invalidates habit-logs + habits queries
  !’ Streak recalculated on next GET /habits/:id/streaks
```

Goal milestone toast

```
useGoalMilestoneTracker(goals) runs on every goals query result
  !’ For each quantitative goal, compute pct = currentValue / targetValue * 100
  !’ Load seen milestones from localStorage
  !’ For each [25, 50, 75, 100] threshold not yet in seen array:
      if pct >= threshold AND initialised ref is true:
          fire Sonner toast with milestone message
          add threshold to seen array
          save to localStorage
  !’ initialised ref is set true after the first pass (prevents spam on page load)
```

Weekly review data fetch

```
GET /api/dashboard/weekly-review
  !’ Server computes rolling 7-day window ending today
  !’ Parallel queries: tasks, habits, habit_logs, mood_logs, focus_sessions, goals
  !’ Per-day breakdown: filters each dataset to that date
  !’ Aggregates: total focus minutes, tasks completed, mood average, habit rate
  !’ Weighted weekly score computed (habit 40% + focus 30% + mood 30%)
  !’ Top habit: habit with longest consecutive streak as of today
  !’ Returns full WeeklyReview JSON in a single response
```

11. Development Workflow

Starting services

```
# API server (port assigned by $PORT, proxied at /api)
pnpm --filter @workspace/api-server run dev

# Frontend (port assigned by $PORT, proxied at /)
pnpm --filter @workspace/flowdeck run dev
```

Do not run pnpm dev at the workspace root — individual artifacts need the PORT and BASE_PATH env vars wired by the workflow config.

Typechecking

```
# Full typecheck (builds composite libs first, then checks all packages)
pnpm run typecheck

# Build composite libs only (needed after schema or spec changes)
pnpm run typecheck:libs

# Check a single package
pnpm --filter @workspace/flowdeck run typecheck
pnpm --filter @workspace/api-server run typecheck
```

After changing the DB schema (lib/db/src/schema/)

```
pnpm --filter @workspace/db run push   # push changes to dev DB
pnpm run typecheck:libs                # rebuild lib declarations
```

After changing the OpenAPI spec (lib/api-spec/openapi.yaml)

```
pnpm --filter @workspace/api-spec run codegen   # regenerate hooks + validators
pnpm run typecheck                              # verify everything still compiles
```

Build

```
pnpm run build   # typecheck + build all packages
```

12. Environment Variables

Variable	Required	Description
|D|A|T|A|B|A|S|E|_|U|R|L|	|Y|e|s|	|P|o|s|t|g|r|e|S|Q|L| |c|o|n|n|e|c|t|i|o|n| |s|t|r|i|n|g|
|S|E|S|S|I|O|N|_|S|E|C|R|E|T|	|Y|e|s|	|S|e|c|r|e|t| |f|o|r| |s|e|s|s|i|o|n| |s|i|g|n|i|n|g|
|C|L|E|R|K|_|S|E|C|R|E|T|_|K|E|Y|	|Y|e|s|	|C|l|e|r|k| |s|e|r|v|e|r|-|s|i|d|e| |s|e|c|r|e|t| |k|e|y|
|V|I|T|E|_|C|L|E|R|K|_|P|U|B|L|I|S|H|A|B|L|E|_|K|E|Y|	|Y|e|s|	|C|l|e|r|k| |p|u|b|l|i|s|h|a|b|l|e| |k|e|y| |(|f|r|o|n|t|e|n|d\)|
|P|O|R|T|	|I|n|j|e|c|t|e|d| |b|y| |w|o|r|k|f|l|o|w|	|P|o|r|t| |t|h|e| |s|e|r|v|i|c|e| |s|h|o|u|l|d| |b|i|n|d| |t|o|

Secrets are managed via Replit's secret store — never hard-coded or committed.

Last updated: May 2026